```
        services.AddQuartz(q =>
        {
            q.SchedulerId = "AUTO";                                    ❸

            q. UseMicrosoftDependencyInjectionJobFactory();

            q.UsePersistentStore(s =>                                  ❹
            {
               s.UseSqlServer(connectionString);                       ❺
               s.UseClustering();                                      ❻
               s.UseProperties = true;                                 ❼
               s.UseJsonSerializer();                                  ❼
            });

            var jobKey = new JobKey("Update_exchange_rates");
            q.AddJob<UpdateExchangeRatesJob>(opts =>
                opts.WithIdentity(jobKey));

            q.AddTrigger(opts => opts
                .ForJob(jobKey)
                .WithIdentity(jobKey.Name + " trigger")
                .StartNow()
                .WithSimpleSchedule(x => x
                    .WithInterval(TimeSpan.FromMinutes(5))
                    .RepeatForever())
            );
        });

        services.AddQuartzHostedService(
            q => q.WaitForJobsToComplete = true);
    })
    .Build();
host.Run();
```

❶ Configuration is identical for both ASP.NET Core apps and worker services.

❷ Obtains the connection string for your database from configuration

❸ Each instance of your app must have a unique SchedulerId. AUTO takes care of this for you.

❹ Enables database persistence for the Quartz.NET scheduler data

❺ Stores the scheduler data in a SQL Server (or LocalDb) database

❻ Enables clustering between multiple instances of your app

❼ Adds the recommended configuration for job persistence

With this configuration, Quartz.NET stores a list of jobs and triggers in the database, and uses database locking to ensure that only a single instance of